

TASK1QA_Abdallah_Ismail

Capatcha Automation Questions

Q1: Explain how you improve the score or lower it, mention the parameters.

The reCAPTCHA v3 engine calculates a score (from 0.0 to 1.0) based on the perceived "risk" of a user session. A high score (e.g., 0.9) indicates a trusted, human-like user, while a low score (e.g., 0.1) suggests a bot. The score is determined by a wide range of parameters that profile the user's environment and behavior.

Improving the score involves making the automated browser session appear as human and legitimate as possible. Lowering the score happens when the session exhibits bot-like characteristics.

Here is a breakdown of the key parameters and their impact:

Parameter / Factor	How it Improves Score (Appears Human)	How it Lowers Score (Appears Bot-like)
IP Address Reputation	Using clean, residential, or high-quality datacenter proxies. An IP with a history of legitimate traffic is trusted.	Using flagged, public, or low-quality datacenter proxies. IPs known for spam or scraping are heavily penalized.
Browser Fingerprint	A consistent and common browser profile: standard User-Agent, viewport size, language (en-US), and timezone. Using <code>selenium-stealth</code> or <code>playwright-stealth</code> helps mask automation flags.	An inconsistent or unusual fingerprint. For example, a rare User-Agent, missing plugins, or detectable automation flags (like <code>navigator.webdriver</code>).

Parameter / Factor	How it Improves Score (Appears Human)	How it Lowers Score (Appears Bot-like)
Behavioral Analysis	Mimicking human interaction: realistic mouse movements (curved paths, variable speed), natural scrolling patterns, and human-like delays between actions.	Instantaneous, linear mouse movements. No scrolling or robotic, fixed-delay scrolling. Actions are performed too quickly.
Session History & Cookies	A "warmed-up" browser profile with a history of visiting legitimate sites (like Google, news sites) and having relevant cookies (especially Google cookies). This signals a real user.	A brand-new, sterile browser session with no history or cookies. This is a major red flag for a bot.

Parameter / Factor	How it Improves Score (Appears Human)	How it Lowers Score (Appears Bot-like)
Headless Mode Detection	Running in headed mode or using advanced headless stealth techniques (like those implemented with CDP overrides) to mask headless-specific browser properties (e.g., WebGL renderer, plugin info).	Running a standard headless browser, which has a distinct fingerprint that is easily detected by reCAPTCHA.
Environment Consistency	Ensuring all browser properties are consistent. For example, the timezone, geolocation, and language should align with the proxy's location and the User-Agent's typical user base.	Mismatched properties. For example, a US-based IP address with a Russian language setting and a Chinese timezone is highly suspicious.

Q2: Research Recaptcha V3 and answer the following

A: What are the different types of recaptcha v3, if any.

State the differences & make a Parameter-Issue-Solution report for each type to solve it.

Unlike reCAPTCHA v2, **reCAPTCHA v3 does not have different "types"** (like the "I'm not a robot" checkbox or invisible challenges). It is a single, unified system that works invisibly in the background to generate a risk score for each user session.

The only significant variation in its implementation is the use of **action tags**. Developers can specify an `action` (e.g., `login`, `submit_form`, `homepage`) when calling `grecaptcha.execute`. This tag does not change how reCAPTCHA v3 works but provides the site owner with a more detailed breakdown of traffic risk in their admin console.

Since there is only one type, the following Parameter-Issue-Solution report applies to all reCAPTCHA v3 implementations:

Parameter / Factor	Issue	Solution
IP Reputation	The IP address is from a known datacenter or has a history of abuse, leading to an immediate low score.	Use high-quality, rotating residential or mobile proxies to mimic legitimate user origins.

Parameter / Factor	Issue	Solution
Browser Fingerprint	The browser exhibits clear signs of automation (<code>navigator.webdriver</code> is true, inconsistent User-Agent, missing plugins, incorrect screen resolution).	Use stealth-enabled automation frameworks (<code>undetected-chromedriver</code> , <code>playwright-stealth</code>) and CDP overrides to mask automation flags and present a consistent, human-like browser profile.
Behavioral Analysis	Actions are performed robotically: instantaneous mouse movements, no scrolling, and unnaturally fast clicks.	Simulate human behavior: generate curved mouse paths, scroll naturally, and add randomized delays between actions to mimic reading and hesitation.

Parameter / Factor	Issue	Solution
Session History	The browser session is "sterile" — it has no cookies, no browsing history, and has never visited common websites.	"Warm-up" the browser session by visiting legitimate, high-traffic sites (like Google, news sites) to accumulate realistic cookies and history before approaching the target site.
Headless Detection	Standard headless browsers have a distinct fingerprint (e.g., unique WebGL renderer, missing GPU data) that is easily detected.	Use advanced headless stealth techniques, such as injecting scripts via CDP to override headless-specific properties and spoofing GPU/renderer information.

B: What are the two ways to inject tokens?

Once a reCAPTCHA token is successfully obtained (either through solving or a third-party service), it must be submitted with the form to be validated. There are two primary methods to accomplish this:

1. **DOM Injection (Callback Simulation):** This is the most common and reliable method. The token is injected directly into the hidden `<textarea>` element that reCAPTCHA creates on the page. This element usually has the ID `g-recaptcha-response`. After injecting the token, the form is submitted as a normal user would. This method is effective because it mimics the final step of the standard reCAPTCHA flow.

```
// Example of DOM Injection
document.getElementById('g-recaptcha-
response').innerHTML = "TOKEN_HERE";
```



2. **Direct Callback Execution:** Some websites define a global JavaScript function that is executed automatically when a reCAPTCHA challenge is successfully completed. If this function's name is known (e.g., `onSubmit`, `verifyCaptcha`), it can be called directly from the browser's console or via an automation script, passing the token as an argument. This bypasses the need for DOM manipulation but is less common and requires reverse-engineering the site's specific implementation.

```
// Example of Direct Callback Execution
// Assumes the callback function is named
'myCallback' and is globally accessible
myCallback("TOKEN_HERE");
```

Resources

- [ScrapingBee - Web Scraping Basics](#)
- [2Captcha - Solving reCAPTCHA v3](#)

